SPRING FRAMEWORK

DAY 1 — WHY DOES SPRING BOOT EVEN EXIST?

CLIENT — SERVER

EXAMPLE: TYPE WWW.AMAZON.IN IN THE BROWSER → YOUR BROWSER TALKS TO AMAZON'S SERVER.

WHO CAN BE A CLIENT?

- → BROWSER
- → MOBILE APP
- → REACT FRONTEND
- → ANDROID / IOS APP
- → POSTMAN
- → ANOTHER SERVER

CLIENT → ASK
SERVER → RESPOND

HTTP

HTTP = HYPER TEXT TRANSFER PROTOCOL → THE RULEBOOK THAT CLIENT & SERVER BOTH FOLLOW.

IT DECIDES:

- → REQUEST STRUCTURE?
- → RESPONSE STRUCTURE?
- → WHICH METHOD? GET, POST, PUT, PATCH, DELETE
- → HOW WILL DATA BE SENT?

HTTPS = THE SECURE VERSION OF HTTP.

HTTP REQUEST

A REQUEST HAS 4 PARTS:

- → METHOD NAME
- → URL / PATH
- → HEADERS
- → BODY

```
POST /LOGIN
HOST: WWW.CODERARMY.IN
CONTENT-TYPE: APPLICATION/JSON
{
  EMAIL: ------
  PASSWORD: ------
}
```

SIMPLE REQUEST, NO BODY NEEDED:

```
GET /COURSES
HOST: WWW.CODERARMY.IN
```

HTTP RESPONSE

A RESPONSE HAS 3 PARTS:

- → STATUS CODE (200 OK)
- → HEADERS
- → BODY

```
200 OK
CONTENT-TYPE: APPLICATION/JSON
{
  MESSAGE: "LOGIN SUCCESSFUL"
}
```

HTTP METHODS

METHOD	WHAT IT DOES	EXAMPLE
GET	READ DATA	FETCH ALL ORDERS
POST	CREATE DATA	PLACE A NEW ORDER
PUT	REPLACE EVERYTHING	UPDATE FULL PROFILE
PATCH	UPDATE A PART	CHANGE ONLY PHONE NO.
DELETE	REMOVE DATA	CANCEL AN ORDER

STATUS CODES

200	OK — SUCCESS
404	NOT FOUND
500	INTERNAL SERVER ERROR

CORE JAVA

WRITE ONCE, RUN ANYWHERE — SAME BYTECODE RUNS ON WINDOWS, MAC, LINUX (JVM NEEDED).

NORMAL PROGRAM VS WEBSITE

CODE FLOW

START
RUN
STOP
EXIT

WEBSITE FLOW

START
RUN
WAIT FOR REQUEST
GIVE RESPONSE
KEEP RUNNING ↻

A WEB SERVER MUST STAY ALIVE → LIKE `WHILE(TRUE) { ... }`

CORE JAVA ≠ WEB

JAVA KNOWS

→ OBJECTS
→ CLASSES
→ INHERITANCE
→ OOPS

JAVA DOESN'T KNOW

→ HTTP REQUEST
→ HTTP RESPONSE
→ URL
→ HEADERS & BODY

JAVA.NET (SINCE JAVA 1.0)

HAS `SOCKET` AND `SERVERSOCKET` → CAN OPEN A PORT.

```
SERVERSOCKET SERVER = NEW SERVERSOCKET(8080);
```

BUT THE BROWSER'S `GET /COURSES` REACHES JAVA AS RAW BYTES. SO WE MUST DO EVERYTHING MANUALLY:

1. READ INPUT STREAMS
2. PARSE THE REQUEST MANUALLY
3. MAP A METHOD TO AN ENDPOINT (BIG IF / ELSE)
4. BUILD THE HTTP RESPONSE MANUALLY
5. HANDLE MULTI-THREADING

THIS IS BORING, REPEATED WORK → WE NEED A STANDARD SOLUTION.

SERVLET & SERVLET CONTAINER

→ SERVLET = SPECIAL JAVA CLASS THAT HANDLES HTTP REQUESTS (1997, JAVA EE)

→ SERVLET CONTAINER = THE SERVER THAT RUNS SERVLETS

EXAMPLES: TOMCAT ✓ JETTY UNDERTOW

SERVLET CONTAINER (TOMCAT) DOES:

→ OPEN A PORT (SAY 8080)

→ READ INCOMING BYTES

→ PARSE THE HTTP REQUEST

→ MANAGE THREADS

→ CREATE THE HTTP RESPONSE

HOW IT FLOWS

TOMCAT GIVES US JAVA-FRIENDLY OBJECTS (REQUEST & RESPONSE) → IT CALLS DOGET() / DOPOST() ON OUR SERVLET.

SO WHY SPRING?

SERVLETS FIXED HTTP HANDLING, BUT BIG APPS GOT PAINFUL:

→ TOO MUCH BOILERPLATE CODE

→ TOO MANY CONFIGURATIONS

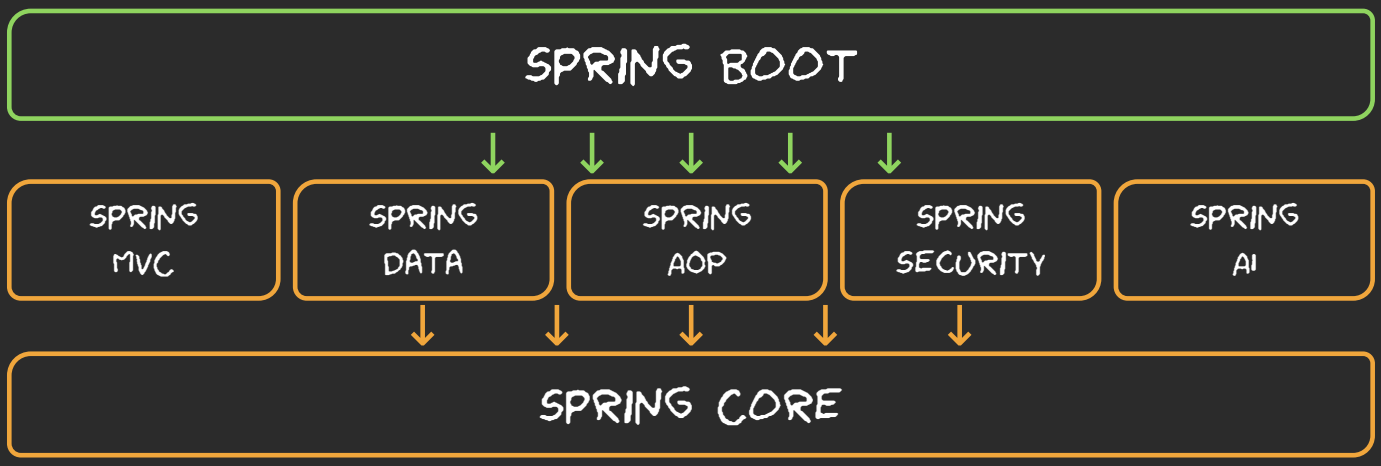
→ TIGHT COUPLING BETWEEN CLASSES

→ HARD TO TEST & MAINTAIN

SPRING FRAMEWORK → MAKES ENTERPRISE JAVA CLEANER

IOC · DEPENDENCY INJECTION · BEANS · LOOSE COUPLING

SPRING = ECOSYSTEM



DI · IOC · BEANS ...

SPRING CORE	FOUNDATION — CREATES & MANAGES OBJECTS
SPRING MVC	BUILD WEB APPS & REST APIS (SERVLETS + CORE)
SPRING DATA	EASY DATABASE ACCESS
SPRING SECURITY	LOGIN, JWT, OAUTH, ROLES
SPRING AOP	LOGGING, TRANSACTIONS (CROSS-CUTTING STUFF)
SPRING AI	ADD AI / LLMS TO JAVA APPS

SPRING DATA & THE DATABASE

JDBC (OLD WAY)

→ WRITE SQL MANUALLY
→ OPEN / CLOSE CONNECTIONS
→ HANDLE RESULT SETS

JPA & HIBERNATE

→ **JPA** = RULES (SPECIFICATION)
→ **HIBERNATE** = DOES THE WORK
→ MAPS JAVA OBJECT ↔ DB TABLE (ORM)

JDBC CRAWLED, HIBERNATE WALKED, SPRING DATA JPA FLIES.

SPRING BOOT

SPRING BOOT IS NOT A REPLACEMENT FOR SPRING.
IT IS AN AUTOMATION LAYER ON TOP OF SPRING.

WHAT IT GIVES US

→ AUTO-CONFIGURATION
→ STARTER DEPENDENCIES
→ EMBEDDED SERVER
→ SENSIBLE DEFAULTS
→ PRODUCTION-READY FEATURES

SPRING	SPRING BOOT
MORE CONFIG	LESS CONFIG
FLEXIBLE	DEFAULTS
FOUNDATION	BUILT ON SPRING

MICROSERVICES

ARCHITECTURE STYLE (NOT A SPRING MODULE) → ONE BIG APP SPLIT INTO SMALL INDEPENDENT SERVICES.

MICROSERVICES = STYLE | SPRING BOOT = TOOL TO BUILD THEM

BIG PICTURE — JAVA FULL-STACK DEV

BROWSER SENDS HTTP REQUEST
↓ SERVLET CONTAINER (TOMCAT) RECEIVES IT
↓ SERVLET HANDLES WEB COMMUNICATION
↓ SPRING MVC → CLEAN WEB PROGRAMMING
↓ SPRING CORE → MANAGES OBJECTS & DEPENDENCIES
↓ SPRING BOOT → AUTO-CONFIGURES EVERYTHING
↓ DEVELOPER WRITES BUSINESS LOGIC

REMEMBER

→ CLIENT ASKS, SERVER RESPONDS — HTTP IS THE RULEBOOK
→ CORE JAVA DOESN'T UNDERSTAND HTTP ON ITS OWN
→ SERVLET + TOMCAT BRIDGE HTTP AND JAVA
→ SPRING CLEANS UP JAVA (IOC, DI) → BOOT AUTO-CONFIGURES IT